UNIVERSITY OF MINES AND TECHNOLOGY (UMaT), TARKWA

DEPARTMENT OF CYBERSECURITY AND INFORMATION SYSTEMS

Echo: Ultrasonic Passwordless Authentication for Web Applications

Boakye Alfred Osei

FCM41.018.99.23

BSc Cyber Security

Final Report

Department of Cybersecurity and Information Systems

Semester 2, Academic Year 2025/2026

Submitted in partial fulfilment of the requirements for the award of Bachelor of Science in Cyber Security

Abstract

Passwords are weak. People forget them, reuse them, and get them stolen through phishing. Newer methods, text codes, email links, QR codes, even passkeys, still make the user do something extra. This project builds and tests Echo, a login system that removes that extra step. A laptop plays a short sound through its speakers, too high-pitched for most people to hear. The user's phone picks up the sound, asks the user to approve the login with a tap, and sends back a digital signature that only that phone can produce. The server checks the signature and logs the laptop in. Nothing secret ever travels through the sound, and the phone's private signing key never leaves the phone. A set of 31 automated tests shows that a recorded and replayed login sound, and a forged signature, are both correctly rejected. A second set of tests, still to be run on real hardware, will check how well the sound carries at different distances and noise levels. Echo works well and resists phishing, but it is not perfect: an attacker who can relay the sound over the internet, instead of being physically close, could still get around it.

Acknowledgements

I want to thank my supervisor, Miss Audrey Asante, for her guidance, patience, and honest feedback while I designed, built, and tested this project. Her comments at every meeting shaped both the direction of the system and how carefully it was tested and written up.

I also want to thank the lecturers in the Department of Cybersecurity and Information Systems at the University of Mines and Technology. What I learned in their courses, on cryptography, web security, ethical hacking, and network security, is used directly in this project. Thank you also to my classmates who lent me their phones and their patience during distance and noise tests.

Finally, thank you to my family and friends for their support this semester.

Dedication

To my friends, who tested Echo again and again and gave me the honest feedback that made it better.

Keywords

- Passwordless authentication
- Acoustic data transmission
- ECDSA public-key cryptography
- WebAuthn and FIDO2
- Near-ultrasonic communication
- Replay attack prevention
- Web application security
- Proximity-based authentication

Contents

List of Figures

List of Tables

Chapter 1
Introduction, Aims and Objectives

1.1 Introduction to Problem

For more than fifty years, the password has been the main way people prove who they are on the web. It has not aged well. People are asked to make up, remember, and regularly change dozens of secret words for accounts they barely use, so they do what any person under that kind of pressure would do: they pick easy passwords, and they use the same one everywhere. The 2024 Verizon Data Breach Investigations Report looked at 10,626 confirmed data breaches and found that stealing or guessing login details was the single most common way attackers got in, the starting point in 38% of cases, with phishing close behind at 15%, and human error playing a part in 68% of all breaches [1]. The problem is not that people are careless. The problem is that passwords ask ordinary users to be perfect guardians of a secret that is easy to steal by fake emails, easy to reuse, and expensive to change, while proving nothing about whether the person typing it is really who they say they are.

The tech industry has answered with a wave of "passwordless" login methods: one-time codes sent by text or email, magic links, QR-code login, and, more and more, passkeys built on a standard called FIDO2/WebAuthn. Each of these fixes part of the problem but leaves another part open. A text message code can still be phished, and it depends on the phone network being secure. A magic link removes the typing but adds a second thing the user must switch to, their email inbox, and now the login is only as safe as that inbox. QR-code login means aiming a camera at a screen, which is awkward. Passkeys get the hard cryptography right, tying a secret key to one specific website, but the user still has to notice a pop-up on their screen, pick the right device, and deal with quirks that differ from one browser or phone to the next. None of these widely used methods let a person log in just by having their trusted phone nearby and giving a quick nod of approval.

This project tries to close that gap. It asks whether a sound that people cannot hear, sent between a laptop and a phone, combined with strong cryptography, can give a login experience with no typing beyond a username, no aiming a camera, and no switching to a second app, while still standing up to the same attacks that make passwords and their replacements weak: phishing, replaying a stolen login attempt, and stealing login details from far away.

1.2 Introduction to Project, Aim and Objectives

Project Aim

The aim of this project is to design, build, and test Echo, a passwordless login system for websites. A laptop and a phone that the user has already registered talk to each other using a sound too high-pitched for most people to hear, backed by public-key cryptography. The project also checks, through real testing, whether this idea is both cryptographically sound and reliable enough for everyday use.

Objectives

1. Look at why passwords fail in practice, and compare the trade-offs made by the passwordless methods already in use (one-time codes, magic links, QR-code login, and FIDO2/WebAuthn passkeys), to explain why Echo is designed the way it is.

2. Build a sound-based communication layer, using a library called `ggwave`, that reliably sends a one-time code from a laptop's speakers to a phone's microphone at 18 to 20 kHz (too high for most people to hear), with a normal audible backup for hardware that cannot play those high frequencies.

3. Build the Echo server: registering users and phones, creating and managing one-time codes, checking digital signatures, sending live updates over WebSocket, and letting users recover their account, all backed by a SQLite database.

4. Build the phone app (a Progressive Web App, meaning it installs from a browser): it listens through the microphone, decodes the sound, shows an approve or deny screen, creates a private signing key that never leaves the phone, and signs login requests.

5. Build a set of automated tests that check, without needing real audio hardware, that a replayed login sound, a forged signature, an expired code, and someone else's phone are all correctly rejected by the server.

6. Test how well the sound actually works in the real world, at different distances (0.3 m, 1 m, 2 m) and in different amounts of background noise (quiet, speech, music), aiming for at least 90% success at 1 m in a quiet room.

7. Build and test a way for users to recover their account so they are never locked out for good, along with basic safety features like limiting repeated attempts.

1.3 Research Questions

The testing in this report is built around four questions:

RQ1. Can an everyday laptop speaker and an everyday phone microphone reliably send a short piece of data at a near-silent pitch (18 to 20 kHz) across normal login distances (0.3 to 2 m) and in real background noise?

RQ2. Does tying that sound to a one-time code and a phone-specific digital signature stop the classic attacks that break weaker passwordless methods, namely replaying a recorded login and faking a signature?

RQ3. How long does the whole process take, from the server creating the code to the laptop being logged in, and does that compare well with other passwordless methods?

RQ4. What risks does a sound-based approach still leave open, and how should those risks be explained honestly in a project like this, rather than glossed over?

1.4 Scope

This project is meant to produce a working system that can be properly tested, not a finished commercial product. Echo includes: signing up with a username and registering one phone; logging in using a one-time code sent as sound; digital signatures and server-side checking; live updates over WebSocket and cookie-based sessions; a rate-limited way to recover an account by email; and the ability to remove a registered phone. It deliberately leaves out: a native phone app (the phone side is a browser-installed app instead, to save time during the semester); strong defences against an attacker relaying the sound over a network rather than being physically present (this risk is discussed, but not solved, since solving it needs hardware control a web browser does not give); production-level account features such as email verification or an admin panel; and letting one account register more than one phone. These choices are explained fully in Section 5.4 as things to build next, not as gaps the report tries to hide.

1.5 Project Justification

Passwordless login is not a small or unusual topic. Major tech companies, browser makers, and an industry group called the FIDO Alliance have spent the last ten years pushing toward exactly this goal, and the rollout of passkeys by Apple, Google, and

Microsoft since 2022 shows where the whole industry is heading. But the mainstream solutions are built to fit large companies and specific platforms, which leaves an interesting and useful question for a security project: what is the smallest possible way to prove "the user's trusted phone is here, and its owner approved this login," without depending on one company's infrastructure, and how far can a small, understandable, self-built version of that idea be pushed within one semester? Building the sound channel, the one-time code system, the signing process, and the checking logic from scratch, instead of using an existing WebAuthn library, forces every decision to be explicit and checkable: what is secret, what is public, what expires, and what an attacker can and cannot do with a recording, a stolen signature, or a fake website. That is what makes this a good Cybersecurity capstone project, and it is why this report spends as much space on the threat model and the failed-attack tests as it spends on the working demo.

1.6 Organization of Chapters

The rest of this report has four more chapters. Chapter 2 looks at why passwords fail, compares the passwordless methods already in use, and reviews the sound-based and cryptographic techniques Echo is built on. Chapter 3 explains the step-by-step way the system was built, its overall design, the login process itself, and the technology choices behind each part. Chapter 4 shows the actual design of the server, the laptop page, and the phone app, the automated tests and their results, and the real-world testing of sound reliability and login speed across distance and noise. Chapter 5 draws conclusions against the aims and objectives set out here, states the project's limitations honestly, and suggests what to build next. References and supporting appendices, including the project plan and the full test results, are at the end of the report.

Chapter 2
Literature Review

2.1 Introduction

This chapter looks at the research and industry work that shaped Echo's design. It has three parts. Section 2.2 looks at why passwords keep failing, even after decades of security advice. Section 2.3 compares the passwordless methods used today, using the comparison approach set out by Bonneau and colleagues [6], so the trade-offs are shown clearly rather than just described in general terms. Section 2.4 looks at research on sending data through sound and pairing devices, which is what Echo's sound channel is built on. Section 2.5 brings these three parts together to explain exactly what Echo adds that is new.

2.2 Why Password Authentication Fails

A password asks a user to invent, remember, and correctly recall a made-up secret for every account they own, without ever writing it down somewhere unsafe, reusing it, or being fooled by a fake copy of the login page asking for it. NIST, the US government's standards body, changed its official password guidance for exactly this reason: it moved away from forcing users to change passwords often and follow strict complexity rules, because those older rules pushed people toward predictable passwords that were actually easier, not harder, to guess [3]. Instead, NIST now recommends checking new passwords against lists of already-leaked passwords, allowing long plain-English phrases instead of complex strings, and relying on a second login factor for anything important. In short, even the standards body admits that a password alone is no longer good enough to trust.

The cost of this weakness shows up clearly in real breach numbers. The 2024 Verizon Data Breach Investigations Report found that stolen, reused, or guessed passwords were the leading way attackers broke into systems across 10,626 confirmed breaches, with phishing close behind and human error playing a part in over two-thirds of all cases [1]. OWASP, a well-known security organisation, lists in its Authentication Cheat Sheet everything a developer must add just to keep a normal password login reasonably safe: multi-factor login, protection against stolen-password reuse, limits on repeated attempts, safe cookie settings, and checks against leaked-password lists [2]. Every one of these is a patch added on top of a weak idea, rather than a fix to the

idea itself. That is the real reason the industry has spent the last ten years building alternatives.

2.3 Passwordless Authentication: A Comparative Review

Bonneau and colleagues [6] built a framework for comparing password replacements on ease of use, ease of adoption, and security, and their main finding still holds true more than ten years later: not one method they studied kept every benefit that ordinary passwords already had. In other words, every passwordless method below trades away something in exchange for something else.

2.3.1 One-Time Codes and Magic Links

Text message and email one-time codes fix the password reuse problem directly, since a stolen password database cannot be reused to log in with a one-time code. But the code still has to pass through something the user reads and retypes, so it can still be phished: a fake login page can simply ask for the code and pass it on to the real site straight away. Text-message codes also depend on the mobile network staying secure, and are open to a well-known attack called SIM-swapping, which is exactly why NIST's guidelines discourage using text messages as a login factor for anything sensitive [3]. Magic links remove the retyping step but add a separate channel (email) that the user has to switch to, so the login is now only as safe as that email account.

2.3.2 QR-Code Login

QR-code login, the pattern made popular by WhatsApp Web and used by many workplace login systems, shows a code on the login screen that an already-logged-in phone scans to approve the session. This removes typing completely and ties the approval to a device the user already trusts, but it means aiming a camera at a screen, which is awkward in some situations. It also has its own phishing risk: if an attacker can copy the real site's QR code onto their own fake page, the code itself does not prove which physical device is actually asking for it.

2.3.3 FIDO2 / WebAuthn Passkeys

The FIDO2 and WebAuthn standards, built jointly by the FIDO Alliance and the W3C, are where the industry is currently heading for passwordless login, and they are the technology behind "sign in with a passkey" on Apple, Google, and Microsoft devices since 2022 [4, 5]. A passkey is a pair of matching digital keys created on the user's own device; the private half never leaves that device (or the company's

secure syncing system), and the browser ties every login attempt to the exact website asking for it. This makes passkeys very hard to phish: a fake copy of a login page hosted on the wrong website simply cannot get a valid login approval, no matter how convincing it looks. This is the strongest of the methods covered here, and Echo borrows its main idea directly: a key pair tied to one device, where the private half never leaves that device.
Passkeys are not perfectly smooth to use, though. The user still has to notice and respond to a pop-up, choose the right device or account when more than one is available, and, when logging in on a new device, often has to scan a QR code anyway, similar to the method described above. Syncing passkeys across a person's Apple or Google account is convenient, but it locks the user into that company's system, and the way passkeys look and behave still differs noticeably between browsers and phones, which is part of why they have not yet fully replaced passwords as a fallback option on most websites.

2.3.4 Biometric and Proximity-Based Approaches

Fingerprint and face recognition on a phone are usually used today as a lock on an existing key, for example, unlocking a passkey, rather than being sent to a remote server directly, since fingerprint data is not something that should ever be sent over the internet or stored elsewhere. Bluetooth- and NFC-based login (used in some workplace systems) is closer to what Echo attempts: it logs a user in because a trusted device is nearby, without needing the user to unlock or tap that device every time after an initial setup. The problem is cost and inconsistency: Bluetooth pairing and distance detection behave differently across phone brands and increasingly require the user to grant special permissions, and NFC needs the two devices to be brought close enough to physically touch, which removes the "just walk up and be recognised" feeling this type of login is supposed to give.

2.4 Acoustic Data Transmission and Device Pairing

Using sound to send data between everyday devices is an old idea, older than the smartphone. Madhavapeddy and colleagues [7] showed that ordinary audio tones could carry useful data between two mobile phones during an active phone call, sending around 20 bits per second with a very low error rate over 3 metres, using DTMF-style tones (the same kind used for phone keypads). Their work showed that sound is a genuinely usable wireless channel, not just a novelty, because it needs no Bluetooth pairing, no Wi-Fi connection, and no direct line of sight. That same idea, that ordinary speakers and microphones can carry data with no setup step at login time, is

exactly what makes sound useful as proof that a phone is physically nearby.
Security researchers have also studied sound channels from the opposite direction, as a way data could leak out without permission. Hanspach and Goetz [8] built a network of computers that could pass data to each other, hop by hop, using nothing but their normal speakers and microphones at a near-silent pitch, in the very same 18 to 20 kHz range that Echo uses, and suggested ways to defend against it, such as filtering out high-pitched sound in hardware and watching for unexpected microphone use. Their work is a useful warning for Echo's own threat model (Chapter 4): the same qualities that make a near-silent sound channel convenient for a normal login, it passes through some walls, needs no pairing, and cannot be heard, also make it useful for a malicious data leak on a device that has already been broken into. A security-minded system should be open about when and why it is using the microphone or speaker.
The `ggwave` library [9], which Echo uses to send and receive sound, works by playing several tones at once to represent data (a method called Frequency-Shift Keying) with built-in error correction. It can send roughly 8 to 16 bytes per second depending on the settings used, which is more than enough for a short one-time code and phone identifier. It offers an `ULTRASOUND_NORMAL` setting in the 18 to 20 kHz range, plus a normal audible fallback for hardware that cannot reproduce those high frequencies cleanly, and Echo uses both directly.

2.5 Positioning Echo

Looking at all this research together shows a gap worth filling. The individual pieces Echo needs, a key pair tied to one device (from WebAuthn), a no-setup-needed nearby-device channel (from audio networking research), and an honest understanding of that channel's own risks (from research on sound as a data leak), are each well studied on their own, but rarely put together into one working, understandable website login system outside of the big tech companies' own passkey systems. Echo does not invent a new cryptographic method; it uses the exact same ECDSA signing that WebAuthn uses. What Echo adds is a complete, understandable login process that replaces the device-discovery step passkeys normally handle behind the scenes with a simple sound-based handshake, judged honestly against both what it proves cryptographically (Chapter 4, Section 4.2) and how reliable the sound channel actually is in the real world (Chapter 4, Section 4.3; Chapter 5).

Chapter 3
Methodology

3.1 Development Methodology

Echo was built using **iterative prototyping**. This means each part of the system was built on its own, tested on its own, and only then connected to the rest, instead of designing the whole system on paper first and building it all in one go. This approach was chosen on purpose, because Echo's design is a strict chain: sound goes in, the server checks it, and the laptop gets logged in. A problem in any one part would block everything after it. Checking each part before building on top of it kept problems small and easy to find. The seven build stages, in the order they happened, were:

Stage 1. Does the sound idea even work? Before writing any server code, a simple test page (`tests/ultrasonic-auth-test.html`) was built to answer one question: can a real laptop speaker and a real phone microphone actually send a signal at 18 to 20 kHz (too high-pitched for most people to hear), and at what distance does it stop working? This test also produced the first written list of what the server and phone needed to do.

Stage 2. Build the server first. The backend was built before any screen the user would see, because the screens needed something to talk to. The full database was set up (users, phones, login attempts, recovery codes, login history), and every part of the API was written and tested as it was added, along with the live-update connection.

Stage 3. Build the sound layer and the phone app. With the server ready, the laptop login page was built to connect to `ggwave` (the sound library) and send the one-time code. Then the phone app was built. It listens through the microphone, decodes the sound, and shows an approve or deny screen. Both sides were tested together in the same room first, then at increasing distances. A quiet background check was added so the phone can confirm, before showing anything to the user, that a decoded code actually belongs to the right account.

Stage 4. Add the cryptography. Once the sound channel worked reliably, the sign-up process was built so the phone creates a private signing key and registers the matching public key with the server. The phone's signing code was written, and the server's signature-checking code was finished.

The whole login process was then tested from start to finish: type username, hear sound, tap approve, get logged in. The automated tests were expanded to cover attack scenarios directly.

Stage 5. Add extra security and polish. With the main flow working, the remaining features were added: a rate-limited way to recover an account, the ability to remove a registered phone, a one-time claim step so only the right laptop gets logged in, a live status display on the login page (transmitting, then phone heard it, then approved), and a normal audible backup for hardware that cannot play very high-pitched sound.

Stage 6. Test how well it actually works. A test tool was built to run controlled trials, recording whether the phone picked up the code, whether the login finished, how long it took, the distance, and how noisy the room was, across a range of distances and noise levels (Section 4.3).

Stage 7. Write it up. The final stage produced the API reference, the description of how the system is built, the test results, and the security discussion presented in this report.

While building the system, the account-recovery method originally planned (numeric one-time recovery codes) was replaced with a rate-limited email link instead, and the sign-up process was extended with a QR-code scanner on the phone to make it faster. Both of these changes are normal for iterative prototyping: a working system shows up usability problems that a plan on paper does not. Both changes are discussed further in Section 4.1.4.

3.2 System Architecture

Echo has three parts that work together: a *laptop page* (the device being logged into, running in a normal web browser), a *phone app* (the trusted, registered device, installed from a browser), and a *server*, built with Node.js, that sits between them and stores all the data. Figure 3.1 shows the full login process, step by step.
The technology was chosen to keep things simple and quick to build, not to handle millions of users. Node.js with Express handles the web requests; SQLite, using a feature already built into Node.js, stores users, phones, login attempts, recovery codes, and login history, without needing a separate database program running; a small library called `ws` handles the live WebSocket connection; both the laptop and phone pages are plain JavaScript with no front-end framework, since neither one needs enough on-screen state management to justify one; and `ggwave`, which runs in

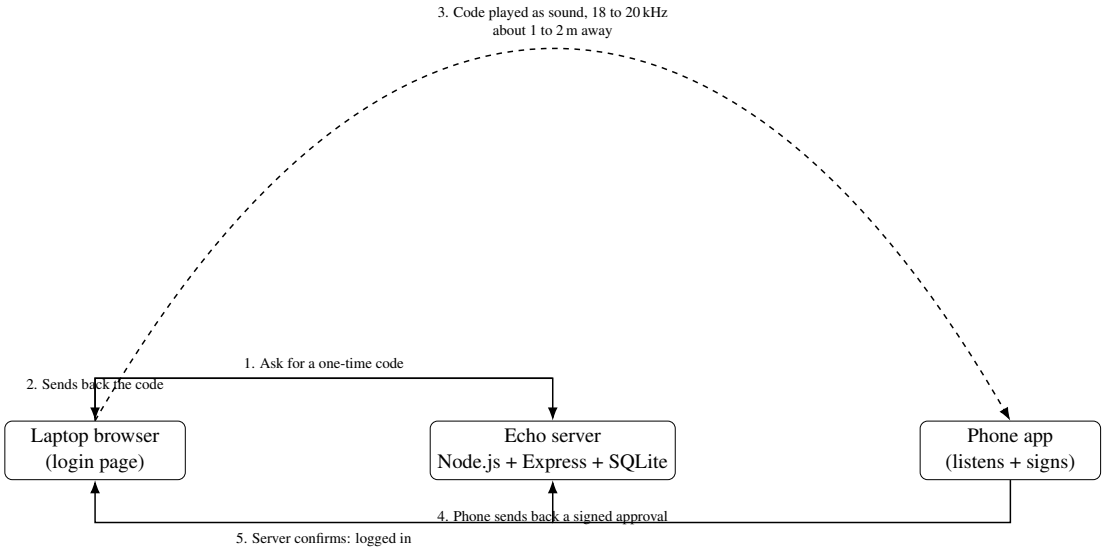

Figure 3.1: The Echo login process: the server creates a one-time code, the laptop plays it as sound, the phone signs it once the user approves, and the server tells the laptop it is logged in.

the browser, is what turns data into sound and back again on both ends. Table 3.1 lists the technology used at each layer.

3.3 How the Login Process Works

3.3.1 Signing Up

Signing up only happens once. The user picks a username (`POST /api/signup`) and gets back a one-time sign-up code. The phone, once it has that code (at first typed in by hand, later scanned as a QR code, see Section 4.1.4), creates a private signing key and a matching public key, right there in the browser, and sends the public key to the server along with the sign-up code (`POST /api/enroll`). The server checks that the key is the right type, marks the sign-up code as used, and saves the public key against the user's account. The private key is created in a way that cannot be copied out, so it only ever exists inside the phone's browser and can never be read, exported, or leaked out by any code, including Echo's own.

3.3.2 Logging In

1. The user types their username on the laptop. The server creates a random one-time code (a "nonce") tied to that one login attempt (`POST /api/login/start`), good for 30 seconds and usable only once.

Table 3.1: Technology used at each layer

Layer	**Technology**
Server	Node.js version 22.5 or later, with Express 4
Database	SQLite, built into Node.js (WAL mode for reliability)
Live updates	`ws` (WebSocket)
Sound sending/receiving	`ggwave`, running in the browser
Cryptography	WebCrypto (on the phone); Node's built-in crypto tools (on the server)
Phone key storage	IndexedDB, a browser storage area the key cannot be copied out of
Phone install	Web App Manifest + Service Worker (lets it work offline)
Web pages	Plain JavaScript, HTML, CSS, no framework
Hosting	Render.com, set up through a `render.yaml` file

2. The laptop turns the code into sound using `ggwave` and plays it at 18 to 20 kHz. If nothing happens, it automatically tries again, up to three times.

3. The phone, always listening, picks up the code and quietly checks with the server that this code belongs to an account it is actually registered to, before showing anything on screen. This stops a phone from ever asking its owner to approve someone else's login.

4. If that check passes, the phone shows an approve or deny screen. If the user approves, the phone signs a short message containing the code and its own device ID, using its private key, and sends that signature to the server (`POST /api/login/verify`).

5. The server looks up the phone's saved public key, checks that the signature matches, checks the code has not expired or been used before, and if everything checks out, marks the code as used and sends a message over the live connection telling the laptop it is approved, along with a one-time claim token.

6. The laptop swaps that claim token for a proper login cookie (`POST /api/session/claim`), which is set with safety flags (`HttpOnly`, `SameSite=Strict`, and `Secure` in production). The claim token can only be used once, so only the laptop that started this exact login can use it.

The exact message the phone signs, `echo-v1|<code>|<deviceId>`, matters more than it looks. Starting it with a version label means the system could change later without confusion, and including the phone's own device ID, not just the one-time code, stops a signature made for one phone from being reused against a different

phone's record, even if an attacker somehow got hold of it. Table 3.2 lists the rules the one-time code follows, and every one of these rules is directly checked by the automated tests in Section 4.2.

Table 3.2: Rules for the one-time login code

Property	**Rule**
Randomness	A random 96-bit value, picked by the server
Time limit	Valid for 30 seconds
Number of uses	Works once only; marked as used right after
Scope	Tied to one login attempt and one username
Signed message	`echo-v1|<code>|<deviceId>`, ties the code to one specific phone
How it travels	Not secret: sent as sound (laptop to phone), then again as plain text when the phone checks in. This is safe because the code can only be used once

3.4 Why This Design Is Secure

Three design choices do most of the security work here, and each one maps to a real decision in the code. First, *the one-time code is public but can only be used once*, so recording and replaying the login sound gets an attacker nothing: the first successful check uses up the code, and the server rejects any attempt to use it again, which is tested directly in Section 4.2. Second, *the private key never leaves the phone*: because it is created in a way that cannot be exported, there is no way, not even through a bug in Echo's own code, to read the raw key back out of the browser. Third, *the signature always goes straight from the phone to the real Echo server, never through the sound itself or through whatever page the laptop happens to be showing*. This is what stops a fake copy of the login page from working: even if a victim's laptop is pointed at a fake site playing the same sound, the victim's phone still only sends its signature to the real Echo server, because that destination is decided by the phone's own code, not by the sound it heard. A fake site can record and replay the sound, but it cannot make the victim's phone send its signature anywhere else. Distance adds a fourth, weaker layer of protection: because an 18 to 20 kHz sound fades quickly in air and does not travel well through walls, an attacker outside the room cannot trigger the sound part of the process at all. Chapter 5 explains why this is a helpful sign of nearby presence rather than a cryptographic guarantee, and how that is different from a proper defence against a relay attack.

3.5 How the Testing Was Planned

Two separate testing tools were built, because there are two different things that need to work: the cryptography needs to be correct, and the sound channel needs to be reliable in the real world. Correctness was checked with a 31-check automated test suite (Section 4.2) that talks to the server directly over the network, using real signing keys but no actual audio hardware, so it runs in seconds and can be repeated after every change without needing a quiet room. Sound reliability was checked separately, by hand, with controlled trials (Section 4.3) at three distances (0.3 m, 1 m, 2 m) and in three noise conditions (quiet, speech, music), with at least 30 trials for each combination, recording whether the phone picked up the code, whether the login finished, and how long it took. This matches the target set out in the objectives (Section 1): at least 90% success at 1 m in a quiet room.

Chapter 4
Design, Testing and Evaluation

4.1 Server Design

The server is split into three files, and each one has a single job: `db.js` handles the database and the security helper functions; `server.js` defines the web app and every API route; and `websocket.js` handles the live connection and the function that tells a waiting laptop that its login was approved. Splitting the code this way keeps each file easy to read on its own, rather than having one huge file that does everything.

4.1.1 Database Layout

Table 4.1 lists the seven tables that make up Echo's database. Two of them, `login_sessions` and `enroll_tokens`, are cleaned up the most carefully: both have an expiry time and a used flag, and a background task (Section 3.2) deletes expired, unused rows every 60 seconds so the database does not fill up with old, useless data over time.

Table 4.1: The database tables

Table	What it stores
`users`	Username, an optional password hash (kept for older accounts), optional email, when the account was created
`devices`	One row per registered phone: which user owns it, its name, and its public key
`login_sessions`	One row per login attempt: the one-time code, its status, a one-time claim token, when it expires, whether it has been used
`sessions`	Logged-in session tokens (the cookie value), with an expiry time
`enroll_tokens`	One-time sign-up codes, tied to a username, with an expiry time and a used flag
`recovery_codes`	Scrambled (hashed) versions of one-time recovery codes, from the older recovery method
`logins`	A running log of every login attempt: user, method used, phone, whether it worked, and when
`magic_tokens`	One-time email recovery links, with an expiry time and a used flag

Where a password hash is stored at all, it is scrambled using a standard, slow method called PBKDF2, with a random 256-bit salt and 310,000 rounds, following the cur-

rent minimum recommended by NIST (the body that sets US security standards) for this method. Checking a password always runs the full scrambling process and compares the result carefully, rather than stopping early, so that the time it takes cannot leak any hints about whether a guess was close. Recovery codes and email links are likewise never stored as plain, readable text.

4.1.2 The API

The server offers a set of web addresses (an API) under `/api`, plus one live WebSocket connection. Table 4.2 lists them all; the full details of what each one expects and returns are in Appendix B.

Table 4.2: The API, in brief

Address	**What it does**
`POST /api/signup`	Claim a username, get back a one-time sign-up code
`POST /api/enroll`	Use that code to register a phone's public key
`GET /api/signup/status`	Check whether sign-up has finished yet
`POST /api/login/start`	Create a one-time login code to play as sound
`GET /api/login/check`	The phone's quiet check that a code belongs to its own account
`POST /api/login/verify`	Send the phone's signature; approves the login if it checks out
`POST /api/session/claim`	Swap a one-time claim token for a real login cookie
`POST /api/login/magic-request`	Ask for an email recovery link
`GET /api/login/magic`	Use that email link to log in
`POST /api/device/token`	Get a code to register a second phone (must be logged in)
`POST /api/device/revoke`	Remove a registered phone (must be logged in)
`GET /api/me`	See your profile, phones, and recent logins (must be logged in)
`POST /api/logout`	End the current login session
`WS /ws?session=<id>`	The laptop's live connection, used to hear "you're approved"

4.1.3 The Laptop Page and the Phone App

The laptop login page is a simple web page that asks the server for a one-time code, opens a live connection for that login attempt, and uses `ggwave` to turn the code into

sound and play it out loud (though too high-pitched to actually hear). A status message on screen moves through three steps, sending, phone heard it, approved, so the person watching can tell what stage the login has reached, rather than staring at a blank screen for up to 30 seconds. If nothing happens, the page tries again automatically, up to three times.
The phone app installs itself from the browser onto the phone's home screen, and can even work offline after the first time it is opened. Once open, it asks for permission to use the microphone and listens constantly for a sound signal. When it picks one up, it runs the quiet background check described in Section 3.3 before showing anything on screen, so the phone only ever asks its owner to approve logins for accounts it is actually registered to. Tapping approve makes the phone sign the login using its private key, which is stored safely on the phone and never leaves it, and send that signature straight to the server.

4.1.4 How the Design Changed During the Project

Because Echo was built step by step (Chapter 3), a few details changed from the original plan once the working system showed real problems the plan on paper had missed. Looking back at the project history against the original plan shows three clear changes, listed here honestly rather than left out:

- **How account recovery works.** The original plan called for six one-time numeric recovery codes per account, stored as scrambled hashes. That was built (the `recovery_codes` table and its code are still in the project), but it was replaced as the main recovery method by a rate-limited email link instead, which felt more practical for someone without a pre-printed list of codes on hand. The key safety property stayed the same either way: recovery is rate-limited and cannot be brute-forced.

- **How sign-up works.** The original plan assumed the sign-up code would be typed into the phone by hand. A QR-code scanner was added to the phone app instead, so the laptop can show the sign-up link as a scannable code. This made signing up noticeably faster without changing how the underlying code actually works or how safe it is.

- **Making the live demo more reliable.** Hosting the project on Render.com's free plan brought up a real-world problem the original plan did not consider: the server falls asleep after about 15 minutes with no visitors, so the very first request after that would take up to 30 seconds to load. A small background task was added that pings the live server every few minutes to stop it from falling asleep during demonstrations. It has no effect when running the project locally.

Other work, such as a dark mode toggle, a more polished home page, and consistent styling across the site, was also done during the project but does not change how the login itself works or how secure it is, so it is not covered in detail here.

4.2 Testing

The correctness of the login process was checked using automated tests that talk to a running copy of the server over the network, using real signing keys to both sign and forge messages, but no actual sound hardware, so the whole set of tests runs in seconds and can be repeated after every change. The tests are split into two files: `test-flow.js` checks the whole login process from start to finish, including the attack scenarios that matter most for the security claims in Section 3.3, and `test-cross-talk.js` checks specifically that one user's phone can never be asked to approve a different user's login.
Both files were run again against the current code for this report, using Node.js version 22.22.3, on 8 August 2026. Table 4.3 shows the results. Every check that matters for the security claims in this report, replaying a recording, forging a signature, one user's device being used against another account, expired codes, and reused tokens, all passed.

Table 4.3: Automated test results (Node.js v22.22.3, run locally)

Test file	**Checks**	**Result**
`tests/test-flow.js` (full login process, including replay, forgery, wrong-device, reused claim token, recovery, removing a phone)	22	22 passed, 0 failed
`tests/test-cross-talk.js` (stops one user's phone approving another user's login)	9	9 passed, 0 failed
Total	**31**	**31 passed, 0 failed**

Looking specifically at the threat model in Section 3.3, these attack attempts were tested directly and every single one was correctly blocked: signing up without a valid code; reusing an already-used sign-up code; replaying an already-used login code; a signature made with the wrong private key; a correct signature used against the wrong user's device; reusing a claim token that has already been used once; and trying to log in with a phone that has been removed from the account. Two smaller checks also confirm that only a phone's own account passes its quiet background check, and that a made-up code and an already-used code are correctly told apart with different, clear error messages, which matters for spotting problems quickly during a

live demo.

4.3 Evaluation

4.3.1 How the Real-World Test Was Planned

Section 3.5 set a target of at least 90% success at 1 m in a quiet room, to be tested across nine combinations: three distances (0.3 m, 1 m, 2 m) and three noise levels (quiet, speech, music), with at least 30 attempts for each combination, 270 attempts in total. Each attempt records: whether the phone picked up the code at all, whether the whole login finished successfully, how long it took from start to finish, the distance, and the noise level, matching the format already planned for in the project's evaluation tool (Appendix B).

Table 4.4: Real-world test results, to be filled in (270 attempts: 3 distances times 3 noise levels times 30 attempts each). This table should be filled in with real measurements from the actual laptop and phone before the report is submitted.

Distance	**Noise**	**Attempts**	**Success rate**	**Typical time**
0.3 m	Quiet	30	*to be filled in*	*to be filled in*
0.3 m	Speech	30	*to be filled in*	*to be filled in*
0.3 m	Music	30	*to be filled in*	*to be filled in*
1 m	Quiet	30	*to be filled in*	*to be filled in*
1 m	Speech	30	*to be filled in*	*to be filled in*
1 m	Music	30	*to be filled in*	*to be filled in*
2 m	Quiet	30	*to be filled in*	*to be filled in*
2 m	Speech	30	*to be filled in*	*to be filled in*
2 m	Music	30	*to be filled in*	*to be filled in*

A note on this table. Table 4.4 is left as a template on purpose, not filled in with made-up numbers. How well the sound carries, and how long a login takes, depends on the exact laptop speaker and phone microphone used, the shape and material of the room, and the background noise on the day of testing. None of that can be measured while writing this report on a computer with no speakers or microphone to test with. The plan for collecting this data (recording success, timing, distance, and noise for every attempt) is ready to use; what is left is to actually run the 270 test attempts on the real laptop and phone, fill in this table with the real numbers, and update the conclusion in Section 5.1 to match. This gap is stated plainly here, instead of being hidden behind invented numbers, because made-up results would give a false picture of how well the system actually performs in the real world, which is the whole point of this section.

4.3.2 An Early Sign It Works

Separate from the full 270-attempt test, an early feasibility test (`tests/ultrasonic-auth-test.html`, mentioned in Chapter 3) already showed that the 18 to 20 kHz sound channel works between a real laptop speaker and a real phone microphone at close range. That result is the reason the project moved ahead into full development in the first place. That same test page is still available for quick, informal checks, and it was used to help choose the 0.3 to 2 m distance range and the quiet/speech/music noise levels used in the full test plan above.

Chapter 5
Conclusions and Further Work

5.1 Summary of Findings

This project set out to find whether a sound too high-pitched to hear, combined with strong cryptography, can give a passwordless web login that needs no typing beyond a username, no aiming a camera, and no second app to think about, while still standing up to the attacks that make passwords and their replacements weak. Looking back at the research questions from Section 1:

RQ1 (does the sound actually work). The early feasibility test and the finished build both confirm that an ordinary laptop speaker and an ordinary phone microphone can reliably exchange a short signal at 18 to 20 kHz at close range, which is why the project was able to move forward into full development. What this report cannot yet say with the same confidence is the exact success rate across the full range of distances and noise levels planned in Section 3.5, because, as stated plainly in Table 4.4, the full 270-attempt test had not been carried out on the real presentation hardware at the time of writing. This is the single most important piece of work still to do before the project's performance claim is fully backed up by evidence, and it is said clearly here rather than covered up with invented numbers.

RQ2 (is the cryptography correct). This question can be answered with much more confidence. The 31-check automated test suite (Section 4.2), run again for this report, passed completely: replaying a used code, forging a signature with the wrong key, using a valid signature against the wrong device, reusing a claim token, and trying to log in with a removed phone were all correctly blocked, and the background check correctly stopped one user's phone from ever being asked to approve someone else's login. Within what the tests actually check, the cryptography behaves exactly as designed: nothing secret ever needs to travel through the sound, and a one-time code combined with a device-specific signature makes the classic replay and forgery attacks that break weaker passwordless methods (Chapter 2) effectively impossible against this system.

RQ3 (how fast is it). The full process, creating the code, playing it as sound, decoding it, the background check, user approval, signing, checking, and confirming the session, was designed around an 8 to 10 second target based on the demo script used during development. Exact, measured timing figures depend on the same real-world test described under RQ1, and are left for that completed test rather than guessed at here.

RQ4 (what risks are left). The clearest weakness is a **relay attack**: because the sound channel only shows that a phone is probably nearby, not that it definitely is, an attacker who can stream the sound from the victim's laptop to the victim's phone over the internet, instead of through the air, could trick the system into approving a login without ever being physically close. Echo does not currently defend against this. It was treated from the start as a known, accepted risk rather than a solved problem, because properly solving it needs very precise timing measurements that a web browser simply does not allow a developer to make. This is a real weakness, and it is presented as one, not played down. A second, smaller risk is an unlocked phone that has been stolen and is physically near the victim's laptop: because Echo does not currently require a fingerprint or face check on the phone immediately before signing (a feature that was planned but not finished in time), anyone holding an unlocked, registered phone near the laptop could approve a login. Both of these risks, and the reasoning for treating them as known limitations rather than blockers, are discussed further below.

5.2 Conclusions Against the Objectives

Measured against the seven objectives set out in Section 1, five were fully met: comparing why passwords fail against the existing passwordless options (Chapter 2); the sound-based transmission layer with an audible backup (Section 3.2); the full server, covering sign-up, the one-time code system, signature checking, live updates, and account recovery (Chapter 4); the phone app with its own signing key (Chapter 4); and the automated test suite (Section 4.2), which was not only built but was actually run for this report with a clean 31 out of 31 pass. The seventh objective, account recovery, was met in a changed form: an email link replaced the originally planned recovery-code system as the main way to recover an account, for the reasons given in Section 4.1.4, while the original recovery-code system is still present in the code as a backup option. The sixth objective, testing across distance and noise, is the one genuinely unfinished item: the tool and the plan for doing it are built and ready (Section 3.5), but the actual 270-attempt data collection still needs to happen on the real demonstration hardware. Overall, the project delivers a working system whose cryptography is sound and has been properly tested, but whose real-world sound reliability is not yet backed by finished measurements, and that gap is this report's clearest limitation.

5.3 Limitations

Beyond the relay-attack and unlocked-phone risks already discussed, three more limitations are worth stating plainly. First, once finished, the real-world test will only describe one specific laptop and phone. Speaker and microphone quality varies a lot between different devices, so a result from the development machine should not be assumed to apply to every laptop and phone without further testing. Second, the phone side depends on the browser continuing to allow microphone access and running the app in the background the way the phone's operating system allows; iPhone's Safari browser is known to behave differently around background microphone access, and this was not fully solved within the time available for this project. Third, the system was built and tested as a single-user, single-phone-per-account demo; it does not yet have a proper way to register or remove more than one phone per account beyond the basic version already tested, which would be needed before the design could be considered ready for real, everyday users.

5.4 Recommendations and Further Work

The following are recommended as the next steps, in order of importance:

1. **Finish the real-world sound test.** Run the full 270-attempt test described in Section 3.5 on the real presentation hardware, fill in Table 4.4 with the actual success rates and timing results, and update the RQ1 and RQ3 conclusions above to match the real numbers before final submission. This is the single most important task left.

2. **Add a fingerprint or face check before signing.** Requiring the phone to check the user's fingerprint or face right before it signs a login closes the unlocked-phone risk described above. This was already planned as the next priority but did not make it into this build, and should be the first thing added if the project continues.

3. **Look into simple ways to reduce the relay-attack risk.** A full, certain fix is out of reach of a web browser, but two simpler ideas are worth trying: timing how long the whole process takes as a rough distance signal, and comparing a short recording of background noise from both the laptop and the phone to check they are likely in the same room. Neither would fully solve the problem alone, but either would make a relay attack harder than it currently is.

4. **Support more than one phone per account, with proper management.** The database already allows more than one phone per user; building a proper screen

for registering and removing phones would move the project from a single-user demo toward something closer to a real, usable feature.

5. **Build a proper phone app.** A dedicated Android or iPhone app, using the phone's own secure hardware for storing the signing key, would remove the web browser as a weak point for key storage entirely. Right now the key cannot be exported, but it still lives inside the browser's storage rather than in dedicated secure hardware built for exactly this purpose.

5.5 Closing Remarks

The main idea this project tested, that a nearby, high-pitched sound combined with strong cryptography can be a workable passwordless login method, holds up well on the cryptography side. Its resistance to replay attacks, forged signatures, and phishing is not just designed on paper but has been properly tested and confirmed with a clean, fully passing set of automated tests. Its real-world sound reliability is meant to be tested with the same level of care, and the honest state of this report is that the real-world data needed to fully back that up still needs to be collected. Stating that gap openly fits the wider goal of this project: to make every part of a passwordless login, what is secret, what is public, what expires, and what still needs to be proven, clear and checkable, instead of hidden behind one company's closed system.

References

[1] Verizon, *2024 Data Breach Investigations Report*, Verizon Business, 2024. Available: https://www.verizon.com/business/resources/reports/2024-dbir-data-breach-investigations-report.pdf

[2] OWASP Foundation, *Authentication Cheat Sheet*, OWASP Cheat Sheet Series. Available: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

[3] P. A. Grassi et al., *Digital Identity Guidelines: Authentication and Lifecycle Management*, NIST Special Publication 800-63B, National Institute of Standards and Technology, 2017 (with updates). Available: https://pages.nist.gov/800-63-3/sp800-63b.html

[4] FIDO Alliance, *FIDO2: WebAuthn & CTAP*. Available: https://fidoalliance.org/fido2/

[5] World Wide Web Consortium (W3C), *Web Authentication: An API for Accessing Public Key Credentials Level 2*, W3C Recommendation. Available: https://www.w3.org/TR/webauthn-2/

[6] J. Bonneau, C. Herley, P. C. van Oorschot, and F. Stajano, "The Quest to Replace Passwords: A Framework for Comparative Evaluation of Web Authentication Schemes," in *Proceedings of the 2012 IEEE Symposium on Security and Privacy*, pp. 553 to 567, 2012.

[7] A. Madhavapeddy, R. Sharp, D. Scott, and A. Tse, "Audio Networking: The Forgotten Wireless Technology," *IEEE Pervasive Computing*, vol. 4, no. 3, pp. 55 to 60, 2005.

[8] M. Hanspach and M. Goetz, "On Covert Acoustical Mesh Networks in Air," *Journal of Communications*, vol. 8, no. 11, pp. 758 to 767, 2013.

[9] G. Gerganov, *ggwave: Tiny Data-Over-Sound Library*, GitHub repository. Available: https://github.com/ggerganov/ggwave

[10] B. A. Osei, *Echo: Ultrasonic Passwordless Authentication, Source Code*, GitHub repository, 2026. Available: https://github.com/Alphred-OB/echo

Chapter A
Project Plan

Table A.1 shows the project's plan, phase by phase, matched against the course deadlines it was written to meet.

Table A.1: Project plan and milestones

Phase	**Dates**	**What was delivered**
Phase 0: Feasibility	Before the build started	The `ggwave` test page working on real hardware
Phase 1: Core system	10 to 28 June	Server, one-time code system, laptop sender, phone decoder
Phase 2: Cryptography finished	29 June to 10 July	Signing, checking signatures, live login updates, leading to the first working demo (13 to 17 July)
Phase 3: Hardening	18 July to 10 August	Groundwork for a fingerprint check, audible backup mode, status display, rate limiting
Phase 4: Evaluation	11 to 21 August	Building and running the real-world test, leading to the final demo (24 to 28 August)
Phase 5: Report	22 to 30 August	Writing this report

Fixed course dates: proposal submitted 14 June 2026; proposal presentation 15 to 19 June 2026; first working demo 13 to 17 July 2026; final demo 24 to 28 August 2026; report due 30 August 2026.

Chapter B
API Reference

This appendix lists the exact request and response for every address in Echo's API, first summarised in Table 4.2. Unless stated otherwise, all requests and responses use JSON. All timestamps are in milliseconds. Every address listed here starts with `/api`, except for the WebSocket address.

POST /api/signup

Claims a username and gives back a one-time sign-up code. **Sends:** `{ username: string }` (2 to 32 characters, lower case letters, numbers, underscore, dot, or hyphen). **Replies:** 200 code issued; 400 invalid format; 409 username already taken with a phone already registered to it.

POST /api/enroll

Uses a sign-up code to register a phone's public key. **Sends:** `{ enrollToken, deviceName?, publicKeyJwk }`, where `publicKeyJwk` must be an EC key on the P-256 curve. **Replies:** 200 phone registered, returns a `deviceId`; 400 missing or wrong fields; 401 code invalid, expired, or already used.

GET /api/signup/status?token=...

Checks whether sign-up has finished yet. **Replies:** 200 `{ enrolled, deviceId, expired }`; 404 code not found.

POST /api/login/start

Creates a one-time login code, good for one login attempt only, to be played as sound. **Sends:** `{ username }`. **Replies:** 200 `{ sessionId, nonce, ttlMs }`; 404 username not found, or no phone registered to it.

POST /api/login/verify

Sends the phone's signature over the one-time code it decoded. **Sends:** `{ nonce, deviceId, signature }`. The signed message is `echo-v1|<nonce>|<deviceId>`. **Replies:** 200 signature valid, laptop told over WebSocket, with a `claimToken`; 400 missing fields; 401 code unknown, expired, already used, wrong signature, or wrong device.

POST /api/session/claim

Swaps a one-time claim token, received by the laptop over WebSocket, for the login cookie. **Sends:** `{ sessionId, claimToken }`. **Replies:** 200 session created, cookie set with safety flags (`HttpOnly`, `SameSite=Strict`, and `Secure` in production); 401 not approved yet, or claim token invalid or already used.

POST /api/login/recovery

The older recovery-code login path. Limited to 5 attempts every 15 minutes per account. **Sends:** `{ username, code }` (in `XXXX-XXXX` format). **Replies:** 200 code valid, session cookie set; 401 invalid; 429 too many attempts.

POST /api/login/magic-request

Asks for an email recovery link (the current main recovery method). **Sends:** `{ usernameOrEmail }`. **Replies:** 200 link sent, if the account exists.

GET /api/login/magic?token=...

Uses an email link to log in and sets the session cookie. **Replies:** 302 redirect to the dashboard on success.

POST /api/recovery/generate

(must be logged in) Creates six one-time recovery codes, shown once as plain text, then stored only as scrambled hashes. **Replies:** 200 `{ codes: [...] }`; 401 not logged in.

POST /api/device/token

(must be logged in) Gives a new sign-up code to register a second phone. **Replies:** 200 `{ enrollToken, ttlMs }`; 401 not logged in.

POST /api/device/revoke

(must be logged in) Removes a phone; its key is rejected on every login after that. **Sends:** `{ deviceId }`. **Replies:** 200 removed; 401 not logged in; 404 phone not found, or not owned by this account.

GET /api/me

(must be logged in) Returns the current user's profile, registered phones, and recent login history.

POST /api/logout

(must be logged in) Ends the current session and clears the login cookie.

WS /ws?session=sessionId

The laptop's live connection, opened right after `POST /api/login/start`. The server refuses the connection if the address is wrong, the `session` value is missing, or no matching login attempt exists. Once the login is approved, the server sends:

```
{ "type": "authenticated", "claimToken": "one-time-token..." }
```

Error format

Every error reply looks the same: `{ "error": "a plain description of what went wrong" }`.

How long data is kept

A background task runs every 60 seconds and deletes: login attempts that expired more than 10 minutes ago, logged-in sessions past their 8-hour limit, and unused sign-up codes that expired more than 10 minutes ago.

Chapter C
Automated Test Results: Full Output

The following is the exact, unedited output of both automated test files, run against a locally hosted copy of the current code (Node.js version 22.22.3) on 8 August 2026. This is the raw evidence behind the results shown in Table 4.3.

tests/test-flow.js

```
Echo protocol test against http://localhost:8000

  (check) signup returns enroll token
  (check) enroll WITHOUT token rejected
  (check) enroll with token succeeds
  (check) enroll token single-use
  (check) taken username rejected
  (check) signup status shows enrolled
  (check) login/start returns nonce + session
  (check) valid signature accepted
  (check) REPLAY rejected (nonce single-use)
  (check) FORGED signature rejected
  (check) ANOTHER user's device rejected
  (check) websocket push received
  (check) session claim sets cookie
  (check) /api/me returns user
  (check) claim token single-use
  (check) magic request succeeds
  (check) magic token generated in DB
  (check) magic login redirects to dashboard
  (check) add-device token issued (auth)
  (check) add-device token requires auth
  (check) device revoked
  (check) REVOKED device cannot even start login

22 passed, 0 failed
```

tests/test-cross-talk.js

```
Testing cross-talk prevention against http://localhost:8000

Enrolled Kwame and Kofi (two independent accounts)
  (check) Kofi login starts successfully
  (check) Kwame device pre-flight check is REJECTED with 403
  (check) Mismatched user error message
  (check) Kofi device pre-flight check succeeds with 200
  (check) Succeeding response returns username
  (check) Fake nonce check returns 404
  (check) User C pre-flight check passes
  (check) User C signature verification succeeds
  (check) Used nonce check returns 410

9 passed, 0 failed
```

Every single check passed (31 out of 31, no failures). The full source code referenced in this appendix and in Chapter 4 is available in the project repository, linked below.

Chapter D
Repository and Deployment

- **Source code:** https://github.com/Alphred-OB/echo, the full project code and its commit history.
- **Live demo:** https://echo-n51n.onrender.com, hosted on Render.com's free plan. The server falls asleep after about 15 minutes with no visitors (so the first visit after that may take up to 30 seconds to load), and the free plan does not keep permanent storage, so the database resets every time a new version is deployed. This is a limit of the free hosting plan, not a fault in the system itself.